

# ADR-8 Modelo de almacenamiento multimedia

## Situación

Propuesto

---

## Contexto

Geojana requiere gestionar dos tipos de contenidos pesados: evidencia fotográfica de reportes ciudadanos (RF14, RF18) y un catálogo educativo con fichas de fauna y protocolos de emergencia (RF09, RF10, RF25). Debido a que el sistema opera en zonas rurales, es crítico que el ciudadano pueda consultar la información educativa y capturar reportes con fotos sin depender de una conexión constante a internet.

---

## Decisión

Se decide utilizar el ecosistema integral de Supabase (SaaS) para la gestión multimedia y de identidad, junto con una estrategia de almacenamiento local híbrido en la PWA para garantizar el acceso offline a la información crítica.

### 1. Uso de Supabase Storage y Auth

- Buckets con RLS: Se utilizará Supabase Storage para almacenar imágenes. Se aplicarán políticas de Row Level Security (RLS) para asegurar que la evidencia de reportes sensibles solo sea accesible por las instituciones pertinentes y el administrador.
- Optimización de Imágenes: Para maximizar el rendimiento en redes 3G/4G, se delegará en Supabase la transformación de imágenes a formato .webp mediante parámetros de consulta en la URL, reduciendo el consumo de ancho de banda.
- Identidad Centralizada: Se usará Supabase Auth para gestionar sesiones, permitiendo que el API Gateway (Node.js) valide tokens JWT de forma eficiente antes de autorizar subidas al storage.

### 2. Estrategia Offline (Imágenes y Catálogos)

Para cumplir con la táctica de Redundancia Pasiva (Warm Standby):

- Cache de Catálogos (Service Workers): El Content Service proveerá las fichas de fauna y protocolos. Estos datos se cachearán en el dispositivo mediante Service Workers (configurados vía Vite), permitiendo que el catálogo sea consultable en el bosque o zonas sin señal.
  - Captura Local de Fotos (IndexedDB): Si no hay conexión al emitir un reporte, la PWA guardará la información y la imagen (en formato Base64 o Blob) dentro de IndexedDB.
  - Resincronización de Estado: Una vez detectada la señal, la PWA realizará la subida automática de las imágenes encoladas al bucket de Supabase y los datos al Sighting Service, asegurando una integridad del 99.9%.
- 

## Alternativas Evaluadas

| Criterio          | Supabase (SaaS) | S3 Compatible (MinIO) | Local en VPS     |
|-------------------|-----------------|-----------------------|------------------|
| Costo             | Gratis/Bajo     | Medio                 | Bajo (Incluido)  |
| Carga Operativa   | Nula            | Alta                  | Muy Alta         |
| Integración RLS   | Nativa          | Manual/Compleja       | Manual           |
| Optimización WebP | Nativa          | Requiere Plugin       | Manual (Backend) |

## Consecuencias

- Disponibilidad (+): El ciudadano puede consultar protocolos de emergencia en zonas aisladas gracias al almacenamiento local de catálogos.
- Ahorro de Datos (+): La conversión a .webp y la compresión en el cliente reducen los costos de transferencia para el usuario y el servidor.
- Simplicidad del Backend (+): Los servicios de Python (Sighting y Content) no necesitan procesar flujos de archivos, solo gestionan metadatos y URLs.
- Sobrecarga Operativa Mínima (+): Al evitar el auto-hosteo (vía VPS), el equipo se enfoca exclusivamente en la lógica de negocio.

## Cumplimiento

- Auditoría de Storage: Verificar que el bucket `reporting-evidence` tenga políticas RLS activas vinculadas a los roles del Identity Service.
- Pruebas Offline: Validar que, tras borrar el caché del navegador en modo avión, las guías de emergencia sigan siendo legibles.
- Métrica de Sincronización: Confirmar que los reportes con fotos guardados offline se suban correctamente al recuperar señal sin pérdida de datos.

## Notas

**Autor:** @Valentina Cifuentes Poblete

**Día de publicación:** 04-06-2026

**Última actualización:** 04-06-2026